

M3 – Daten, Routing, REST, Qualität & Backend

Web-Applikationen SS26 · Prof. Dr. Pascal Laube · HTWG Konstanz

Abgabe: nach VL 13 · **VL-Bezug:** VL 05–06, 10–13 (REST, Router, Datenfetching, State, Testing, Architektur, Backend)

Was ihr abgebt

- **Repository-Link** mit aktuellem Stand
 - **README** mit Zuordnung: welches Kriterium ist wo im Code sichtbar
 - Testuser-Credentials ins README oder in eine Abgabenernotiz (keine echten Secrets im Repo)
-

Kriterien

React Router (VL 10)

- Mindestens **2–3 Routen** vorhanden, z.B. Übersicht, Detail, Einstellungen
- Navigation über `<Link>` oder `useNavigate`, kein manuelles `window.location`

Datenfetching & REST (VL 05–06, 10)

- Fetch oder Axios gegen eine API (eigenes Backend)
- Mindestens **GET** (Daten anzeigen) und **eine schreibende Methode** (POST, PUT oder DELETE) implementiert

Fehler- & Ladezustände

- Sichtbarer **Ladeindikator** beim Fetchen (Spinner, Skeleton, Text)
- Sichtbare **Fehlermeldung** bei 4xx/5xx oder Netzwerkfehler (kein stilles Scheitern)

Geteilter State (VL 11)

- App-weiter Zustand über **React Context** (oder vergleichbares Pattern) für mindestens ein Feature, z.B.:
 - Eingeloggter User
 - Warenkorb / Merkliste
 - Theme (hell/dunkel)

Tests (VL 11)

- Mindestens **3–5 Tests** vorhanden
- Empfohlen: Vitest + React Testing Library oder API-Tests mit Supertest
- Getestet wird **Kernlogik oder zentrale UI-Komponenten**, kein reines Smoke-Testing
- Tests laufen mit `npm test` oder dokumentiertem Befehl

Backend (VL 13)

- **Node.js-Server** (Express, Fastify o.ä.) mit **eigenen API-Endpunkten**

- Das React-Frontend spricht dieses Backend an (nicht mehr nur eine externe API)

Datenbank (VL 13)

- **Persistente Datenhaltung:** SQLite, PostgreSQL, MongoDB oder vergleichbar
- Kein `data.json` als Fake-Datenbank

Authentifizierung (VL 13)

- Login und Registrierung implementiert
- **Session oder JWT** konsequent eingesetzt (eine Variante, nicht beide gemischt)
- Mindestens eine **geschützte Route oder ein geschützter Endpunkt** vorhanden

Architektur (VL 12)

- README-Abschnitt mit **Architekturskizze** (SPA + API-Backend, grob)
- Ein Satz Begründung: warum kein SSR/SSG nötig ist (oder warum doch)

Ergebnis

Eine **navigierbare Full-Stack-SPA**, die echte Daten per HTTP von einem **eigenen Backend** bezieht, sauber mit Fehlern umgeht, Zustand über Komponentengrenzen hinweg teilt und mit echten Accounts auf **persistenten Daten** arbeitet. Qualität durch Tests nachgewiesen.

Abgabe-Checkliste

- Vorgestellt
 - getaggt mit "Meilenstein 3"
-

Tipps

- Für eine öffentliche Test-API eignet sich z.B. [JSONPlaceholder](#) oder [The Movie DB](#).
- Fehlerbehandlung: `try/catch` im `useEffect` oder `.catch()` nach `fetch()`.
- Context nicht übertreiben: nur was wirklich app-weit gebraucht wird, gehört rein.
- Startet das Backend früh, bevor ihr alle Frontend-Features fertig habt.
- SQLite mit Prisma oder Drizzle ist ein guter Einstieg ohne Datenbankserver.
- JWT: Token im **Authorization**-Header, nicht im `localStorage` (XSS-Risiko).
- Tests: Testet das, was kaputtgehen kann, nicht das, was offensichtlich funktioniert.
- Keine Secrets im Repository (Datenbankpasswörter, JWT-Secret). `.env`-Datei in `.gitignore`.